OTW_ Wargames

OVERTHEWIRE BANDIT WARGAME

PART 2

Practical Linux & Privilege Escalation Fundamentals for Junior Pentesters

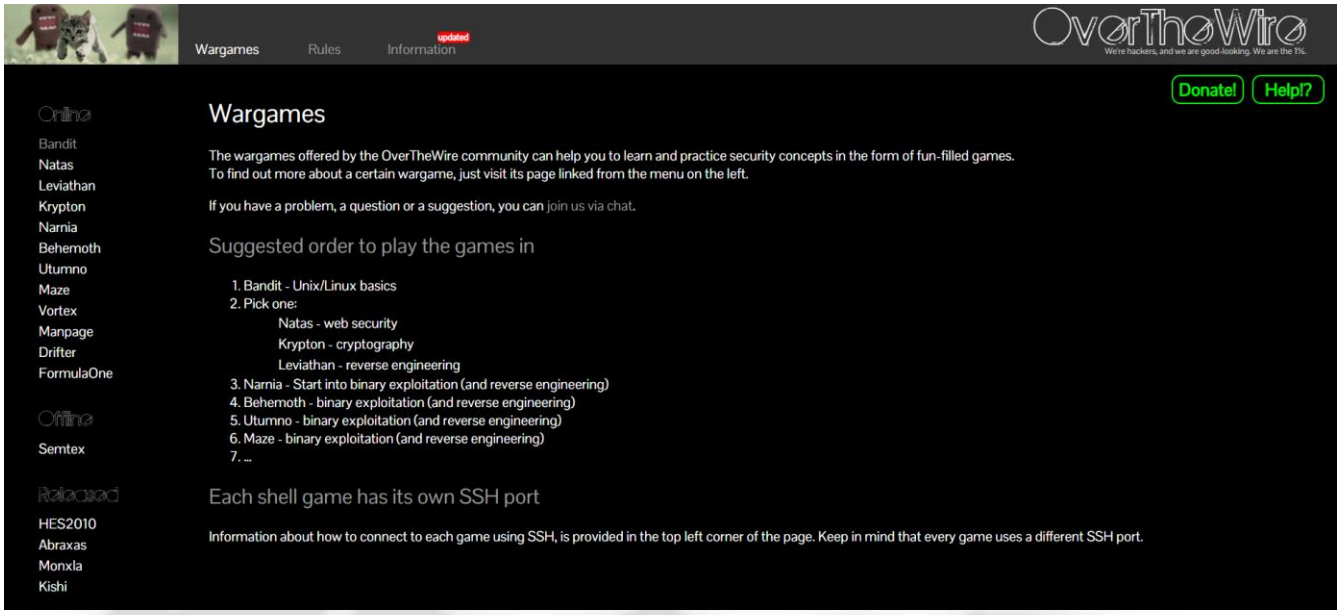
Prepared by

Ahmed Eldkrory (EDGAR)

Junior Penetration Tester

ahmed.a.eldkrory@gmail.com
www.linkedin.com/in/ahmed-eldkrory

Overview



This document demonstrates my understanding of fundamental Linux and command-line concepts from a security perspective, based on the **OverTheWire Bandit Wargame**. It summarizes key techniques, methodologies, and learning outcomes developed while solving the initial levels of the challenge.

The focus of this write-up is on **technical reasoning, problem-solving approach, and practical skill development**, rather than simply listing solutions or commands. Emphasis is placed on understanding how each task reflects real-world scenarios encountered in penetration testing and system analysis.

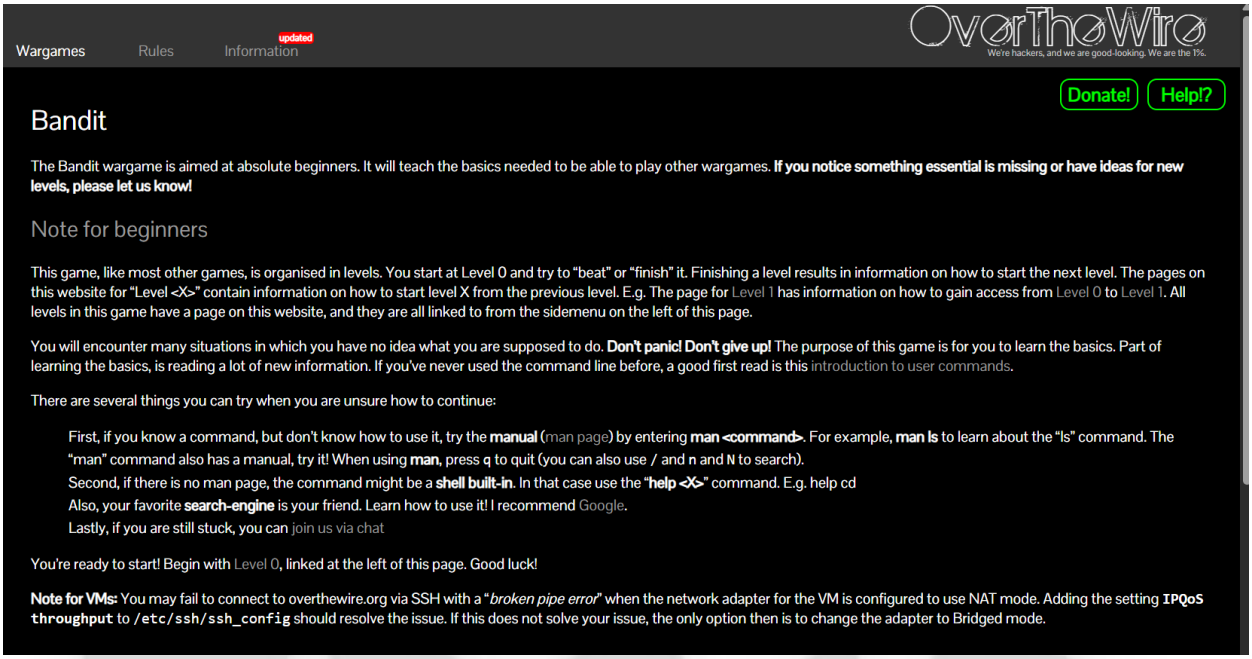
This report specifically covers **Levels 11 → 22** of the Bandit Wargame. Due to the progressive nature of the challenge, the full walkthrough will be divided into **three separate reports**, each addressing a different range of levels to maintain clarity and depth of explanation.

Full Write-up:

https://medium.com/@ahmed.a.eldkrory/from-data-decoding-to-privilege-escalation-overthewire-bandit-levels-11-22-96b64771b33c

Wargame Link:
https://overthewire.org/wargames/bandit/

Bandit Wargame



Wargames Rules Information updated

OverTheWire
We're hackers, and we are good-looking. We are the 1%.

Donate! Help!?

Bandit

The Bandit wargame is aimed at absolute beginners. It will teach the basics needed to be able to play other wargames. **If you notice something essential is missing or have ideas for new levels, please let us know!**

Note for beginners

This game, like most other games, is organised in levels. You start at Level 0 and try to "beat" or "finish" it. Finishing a level results in information on how to start the next level. The pages on this website for "Level <X>" contain information on how to start level X from the previous level. E.g. The page for Level 1 has information on how to gain access from Level 0 to Level 1. All levels in this game have a page on this website, and they are all linked to from the sidemenu on the left of this page.

You will encounter many situations in which you have no idea what you are supposed to do. **Don't panic! Don't give up!** The purpose of this game is for you to learn the basics. Part of learning the basics, is reading a lot of new information. If you've never used the command line before, a good first read is this introduction to user commands.

There are several things you can try when you are unsure how to continue:

First, if you know a command, but don't know how to use it, try the **manual** (man page) by entering **man <command>**. For example, **man ls** to learn about the "ls" command. The "man" command also has a manual, try it! When using **man**, press `q` to quit (you can also use `/` and `n` and `N` to search).
Second, if there is no man page, the command might be a **shell built-in**. In that case use the "**help <X>**" command. E.g. help cd
Also, your favorite **search-engine** is your friend. Learn how to use it! I recommend Google.
Lastly, if you are still stuck, you can join us via chat

You're ready to start! Begin with Level 0, linked at the left of this page. Good luck!

Note for VMs: You may fail to connect to overthewire.org via SSH with a "*broken pipe error*" when the network adapter for the VM is configured to use NAT mode. Adding the setting `IPQoS throughput` to `/etc/ssh/ssh_config` should resolve the issue. If this does not solve your issue, the only option then is to change the adapter to Bridged mode.

The goal of the Bandit Wargame is to introduce beginners to essential Linux skills required in cybersecurity, particularly in penetration testing and system exploitation.

The challenge is structured as a sequence of levels, where each level requires the user to retrieve a password that grants access to the next level. These passwords are typically hidden within files, directories, or system outputs, requiring the use of Linux commands and logical reasoning to uncover them.

Unlike theoretical learning, Bandit emphasizes **hands-on interaction with a real Linux environment**, encouraging users to think like attackers while developing practical command-line proficiency.

For example, a user may need to:

- Navigate complex directory structures
- Identify hidden files
- Read file contents with restricted permissions
- Process and filter command output

Each level builds upon the previous one, reinforcing core skills and introducing new concepts gradually.

Why Bandit Matters in Cybersecurity

Mastering the Linux command line is a foundational requirement in cybersecurity. The Bandit Wargame directly supports this by simulating realistic scenarios that security professionals encounter.

From a penetration testing perspective, these skills are critical for:

- Navigating compromised systems efficiently
- Locating sensitive files and credentials
- Understanding file permissions and access control
- Automating tasks using command-line tools
- Interacting with remote systems via SSH

Additionally, Bandit helps develop a **problem-solving mindset**, which is essential for:

- Privilege escalation
- Post-exploitation activities
- Capture The Flag (CTF) challenges
- Real-world security assessments

By understanding how attackers think and operate at a low level, defenders can better secure systems against similar techniques.

Core Concept: Linux Command-Line Fundamentals

At its core, the Bandit Wargame focuses on mastering the Linux command-line interface (CLI), which is a primary tool for both attackers and defenders.

Key concepts covered include:

- **File System Navigation**
  Understanding how to move through directories and locate files using commands such as `cd`, `ls`, and `pwd`.
- **File Inspection and Manipulation**
  Reading and analyzing file contents using tools like `cat`, `less`, `head`, and `tail`.
- **Permissions and Access Control**
  Interpreting file permissions and understanding how they affect accessibility.
- **Data Processing**
  Filtering and extracting useful information using commands such as `grep`, `sort`, and `uniq`.
- **Remote Access**
  Connecting to systems securely using SSH and managing authentication credentials.

A useful way to think about the command line is as a **direct interface to the operating system**, where every action is explicit and controlled.

Unlike graphical interfaces, the CLI provides:

- Greater precision
- More control over system operations
- The ability to automate complex tasks

These properties make it an essential tool in cybersecurity operations.

Room Structure

The Bandit Wargame is structured as a sequence of progressively challenging levels, each designed to introduce a specific concept or skill.

In this report, the focus is on:

Levels 11 → 22

This stage emphasizes:

- Basic Linux commands
- File handling techniques
- Understanding hidden and non-standard file names
- Introductory data extraction methods

Each level builds incrementally, ensuring a solid foundation before moving to more

advanced topics in later stages.

Operational Tip: Level Transitions

Methodology To progress efficiently, you must close the current session and

re-authenticate as the next user. Avoid nesting SSH sessions,as this creates confusion and connection instability.

Commands

`exit` — Terminate the current session and return to your local terminal.

`ssh bandit[N]@bandit.labs.overthewire.org -p 2220` — Connect to the next level (where [N] is the new level number).

Bandit Level 10 → 11

Donate! Help!?

Bandit Level 10 → Level 11

Level Goal

The password for the next level is stored in the file **data.txt**, which contains base64 encoded data

Commands you may need to solve this level

grep, sort, uniq, strings, base64, tr, tar, gzip, bzip2, xxd

Goal

Retrieve the password from `data.txt`, which is stored as Base64 encoded data.

Technique

Decoding data formats. Base64 is a common encoding scheme used to transmit binary data over media that are designed to deal with textual data. In a security context, it is frequently used to obfuscate strings or encode payloads. Professional analysts must recognize the characteristic padding (e.g., ==) and know how to decode it instantly.

Commands

- `cat data.txt` — Displays the encoded string to verify the format.
- `base64 -d data.txt` — Decodes the file content using the `-d` (decode) flag to reveal the plain-text password.

Result

Password Found: `dtR173fZKb0RRsDFSGsg2RWnpNVj3qRr`

Execution & Documentation

```
bandit10@bandit:~$ cat data.txt
VGhlIHBhc3N3b3JkIGlzIGR0UjE3M2ZaS2IwUlJzREZTR3NnMlJXbnBOVmozcVJyCg==
bandit10@bandit:~$ base64 -d data.txt
The password is dtR173fZKb0RRsDFSGsg2RWnpNVj3qRr
bandit10@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 11 → 12

Donate! Help!?

Bandit Level 11 → Level 12

Level Goal

The password for the next level is stored in the file **data.txt**, where all lowercase (a-z) and uppercase (A-Z) letters have been rotated by 13 positions

Goal

Decrypt the password in `data.txt`, where all letters have been rotated by 13 positions (ROT13).

Technique

Basic Caesar Cipher decryption. ROT13 is a simple substitution cipher that replaces a letter with the 13th letter after it in the alphabet. While not cryptographically secure, it is a classic example of data obfuscation. Security professionals use the `tr` (translate) command to map and shift characters efficiently in the terminal.

Commands

- `cat data.txt` — Views the obfuscated text.
- `cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'` — Maps the first 13 letters to the last 13 (and vice versa) for both upper and lower case to rotate the text back to readable format.

Result

Password Found: `7x16WNeHIi5YkIhWsfFIQoognUTyj9Q4`

Execution & Documentation

```
bandit11@bandit:~$ ls -la
total 24
drwxr-xr-x   2 root     root     4096 Oct 14 09:25 .
drwxr-xr-x 150 root     root     4096 Oct 14 09:29 ..
-rw-r--r--   1 root     root      220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root     root     3851 Oct 14 09:19 .bashrc
-rw-r-----   1 bandit12 bandit11   49 Oct 14 09:25 data.txt
-rw-r--r--   1 root     root      807 Mar 31  2024 .profile
bandit11@bandit:~$ cat data.txt
Gur cnffjbeq vf 7k16JArUVv5LxVuJfsSVdbbtaHGlw9D4
bandit11@bandit:~$ cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
The password is 7x16WNeHIi5YkIhWsfFIqoognUTyj9Q4
bandit11@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 12 → 13

Donate! Help!?

Bandit Level 12 → Level 13

Level Goal

The password for the next level is stored in the file **data.txt**, which is a hexdump of a file that has been repeatedly compressed. For this level it may be useful to create a directory under /tmp in which you can work. Use mkdir with a hard to guess directory name. Or better, use the command "mktemp -d". Then copy the datafile using cp, and rename it using mv (read the manpages!)

Goal

Extract the password from a file that is a hex dump of a repeatedly compressed file.

Technique

Reverse engineering file structures. This level requires multiple steps: converting a hex dump back to binary, identifying file types via headers (signatures), and decompressing multiple layers (Gzip, Bzip2, Tar). This simulates real-world "nested" malware analysis or firmware extraction.

Commands

- `mkdir /tmp/edgar_work` — Creates a dedicated, writable workspace in the `/tmp` directory.
- `xxd -r data.txt > data` — Reverses the hex dump back into its original binary format.
- `file data` — Analyzes the file header to determine the compression type (e.g., Gzipped, Bzip2, or POSIX tar).
- `mv data data.gz` — Renames the file with the correct extension so decompression tools can recognize it.
- `gunzip data.gz` — Decompresses a Gzip layer (repeat logic for `bunzip2` and `tar -xf` as needed).

Result

Password Found: `wbWpNgYkhqBF96WE3f92unEAt6mYpS9X`

Bandit Level 13 → 14

Donate! Help!?

Bandit Level 13 → Level 14

Level Goal

The password for the next level is stored in **/etc/bandit_pass/bandit14 and can only be read by user bandit14**. For this level, you don't get the next password, but you get a private SSH key that can be used to log into the next level. Look at the commands that logged you into previous bandit levels, and find out how to use the key for this level.

Goal

Use a private SSH key to log into the next level and retrieve the password stored in /etc/bandit_pass/bandit14.

Technique

Private Key Authentication. Instead of a password, this level uses an RSA private key for authentication. This is a critical real-world skill, as many cloud environments and servers prohibit password-based logins in favor of more secure key-based authentication. Additionally, it highlights the importance of file permissions; SSH keys must have restricted permissions (600) to be accepted by the client.

Commands

- `ls -la` — Identified the sshkey.private file in the home directory.
- `cat sshkey.private` — Displayed the RSA private key content.
- `chmod 600 bandit14.key` — Set restrictive permissions so the SSH client would accept the key (performed on local machine).
- `ssh -i bandit14.key bandit14@bandit.labs.overthewire.org -p 2220` — Logged into the next level using the -i (identity file) flag.

Result

Password Found: `MU4VWeTyJk8ROof1qqmcBPaLh7lDCPvS` (retrieved after successful login).

Execution & Documentation

```
bandit13@bandit:~$ ls -la
total 24
drwxr-xr-x   2 root     root     4096 Oct 14 09:26 .
drwxr-xr-x 150 root     root     4096 Oct 14 09:29 ..
-rw-r--r--   1 root     root      220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root     root     3851 Oct 14 09:19 .bashrc
-rw-r--r--   1 root     root      807 Mar 31  2024 .profile
-rw-r-----   1 bandit14 bandit13 1679 Oct 14 09:26 sshkey.private
bandit13@bandit:~$ cat sshkey.private
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAxkkOE83W2cOT7IWhFc9aPaaQmQDdgzuXCv+ppZHa++buSkN+
gg0tcr7Fw8NLGa5+Uzec2rEg0WmeevB13AIoYp0MZyETq46t+jk9puNwZwIt9XgB
ZufGtZEwWbFWw/vVLNwOXBe4UWStGRWzgPpEeSv5Tb1VjLZIBdGphTIK22Amz6Zb
ThMsiMnyJafEwJ/T8PQO3myS91vUHEuoOMAzoUID4kN0MEZ3+XahyK0HJVq68KsV
ObefXG1vvA3GAJ29kxJaqvRfgYnqZryWN7w3CHjNU4c/2Jkp+n8L0SnxaNA+WYA7
jiPyTF0is8uzMlYQ4l1Lzh/8/MpvhCQF8r22dwIDAQABAoIBAQC6dWBjhyEOzjeA
J3j/RWmap9M5zfJ/wb2bfidNpwbB8rsJ4sZIDZQ7XuIh4LfygoAQSS+bBw3RXvzE
pvJt3SmU8hIDuLsCjL1VnBY5pY7Bju8g8aR/3FyjyNAqx/TLfzlLYfOu7i9Jet67
xAh0tONG/u8FB5I3LAI2Vp6OviwvdWeC4nOxCthldpuPKNLA8rmMMVRTKQ+7T2VS
nXmwYckKUcUgzoVSpiNZaS0zUDypdpy2+tRH3MQa5kqN1YKjvF8RC47woOYCktsD
o3FFpGNFec9Taa3Msy+DfQQhHKZFKIL3bJDONtmrVvtYK40/yeU4aZ/HA2DQzwhe
ol1AfiEhAoGBAOnVjosBkm7sblK+n4IEwPxs8sOmhPnTDUy5WGrpSCrXOmsVIBUf
laL3ZGLx3xCIwtCnEucB9DvN2HZkupc/h6hTKUYLqXuyLD8njTrbRhLgbC9QrKrS
M1F2fSTxVqPtZDlDMwjNR04xHA/fKh8bXXyTMqOHNJTHHNhbh3McdURjAoGBANkU
1hqfnw7+aXncJ9bjysr1ZWbqOE5Nd8AFgfwaKuGTTVX2NsUQnCMWdOp+wFak40JH
PKWkJNdBG+ex0H9JNQsTK3X5PBMAS8AfX0GrKeuwKWA6erytVTqjOfLYcdp5+z9s
8DtVCxDuVsM+i4X8UqIGOlvGbtKEVokHPFXP1q/dAoGAcHg5YX7WEehCgCYTzpO+
xysX8ScM2qS6xuZ3MqUWAxUWkh7NGZvhe0sGy9iOdANzwKw7mUUFViaCMR/t54W1
GC83sOs3D7n5Mj8x3NdO8xFit7dT9a245TvaoYQ7KgmqpSg/ScKCw4c3eiLava+J
3btnJeSIU+8ZXq9XjPRpKwUCgYA7z6LiOQKxNeXH3qHXcnHok855maUj5fJNpPbY
iDkyZ8ySF8GlcFsky8Yw6fWCqfG3zDrohJ5l9JmEsBh7SadkwsZhvecQcS9t4vby
9/8X4jS0P8ibfcKS4nBP+dT81kkkg5Z5MohXBORA7VWx+AChcDEkprsQ+w32xeD
qT1EvQKBgQDKm8ws2ByvSUVs9GjTilCajFqLJ0eVYzRPaY6f++Gv/UVfAPV4c+S0
kAWpXbv5tbkkzbS0eaLPTKgLzavXtQoTtKwrjpolHKIHUz6Wu+n4abfAIRFubOdN
/+aLoRQ0yBDRbdXMsZN/jvY44eM+xRLdRVyMmdPtP8belRi2E2aEzA==
-----END RSA PRIVATE KEY-----
bandit13@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

```
┌──(edgar㉿kali)-[~]
└─$ nano bandit14.key

┌──(edgar㉿kali)-[~]
└─$ ls -l
total 36
-rw-rw-r-- 1 edgar edgar 1679 Jan 21 19:03 bandit14.key
drwxr-xr-x 2 edgar edgar 4096 Jan 15 17:03 Desktop
drwxr-xr-x 2 edgar edgar 4096 Jan 15 17:03 Documents
drwxr-xr-x 2 edgar edgar 4096 Jan 15 18:52 Downloads
drwxr-xr-x 2 edgar edgar 4096 Jan 15 17:03 Music
drwxr-xr-x 3 edgar edgar 4096 Jan 21 16:09 Pictures
drwxr-xr-x 2 edgar edgar 4096 Jan 15 17:03 Public
drwxr-xr-x 2 edgar edgar 4096 Jan 15 17:03 Templates
drwxr-xr-x 2 edgar edgar 4096 Jan 15 17:03 Videos

┌──(edgar㉿kali)-[~]
└─$ chmod 600 bandit14.key


┌──(edgar㉿kali)-[~]
└─$ ls -la
total 124
drwx------ 17 edgar edgar  4096 Jan 21 19:03 .
drwxr-xr-x  4 root  root   4096 Jan 15 17:00 ..
-rw-------  1 edgar edgar  1679 Jan 21 19:03 bandit14.key
```

```
┌──(edgar㉿kali)-[~]
└─$ ssh -i bandit14.key bandit14@bandit.labs.overthewire.org -p 2220

                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
             More information on http://www.overthewire.org/wargames

backend: gibson-0

      ,----..            ,----,          .---.
     /   /   \         ,/   .`|         /. ./|
    /   .     :      ,`    .'  :      .--'.  ' ;
   .   /   ;.  \   ;    ;     /     /__./ \ : |
  .   ;   /  ` ; .'___,/    ,' .--'.  '   \' .
  ;   |  ; \ ; | |    :     | /___/ \ |    ' '
  |   :  | ; | ' ;    |.';  ; ;   \  \;      :
  .   |  ' ' ' : `----'  |  |  \   ;  `      |
  '   ;  \; /  |      '   :  ;   .   \    .\  ;
   \   \   ',  /       |   |  '    \   \   ' \ |
    ;   :    /        '   :  |     :    '  |--"
     \   \ .'         ;   |.'       \   \ ;
  www. `---` ver      '---' he        '---" ire.org


Welcome to OverTheWire!
```

Bandit Level 14 → 15

Donate! Help!?

Bandit Level 14 → Level 15

Level Goal

The password for the next level can be retrieved by submitting the password of the current level to **port 30000 on localhost.**

Goal

the password for the next level by submitting the current level's password to port 30000 on localhost.

Technique

Network service interaction. This level introduces interacting with local services via specific ports. This simulates a common scenario where a service (like a database or an internal API) is running locally on a compromised machine and needs to be queried to extract further information or escalate privileges.

Commands

- `cat /etc/bandit_pass/bandit14` — Confirmed the current password.
- `nc localhost 30000` — Established a connection to the local service on port 30000 using Netcat (nc).

(After connecting, the current password was pasted and sent to the service).

Result

Password Found: `8xCjnmgoKbGLhHFAZlGE5Tmu4M2tKJQo`

Execution & Documentation

```
bandit14@bandit:~$ cat /etc/bandit_pass/bandit14
MU4VWeTyJk8ROof1qqmcBPaLh7lDCPvS
bandit14@bandit:~$ nc localhost 30000
MU4VWeTyJk8ROof1qqmcBPaLh7lDCPvS
Correct!
8xCjnmgoKbGLhHFAZlGE5Tmu4M2tKJQo

exit
bandit14@bandit:~$ exit
logout
There are stopped jobs.
bandit14@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 15 → 16

Donate! Help!?

Bandit Level 15 → Level 16

Level Goal

The password for the next level can be retrieved by submitting the password of the current level to **port 30001 on localhost** using SSL/TLS encryption.

Helpful note: Getting "DONE", "RENEGOTIATING" or "KEYUPDATE"? Read the "CONNECTED COMMANDS" section in the manpage.

Goal

Retrieve the next password by submitting the current password to port 30001 on localhost using SSL/TLS encryption.

Technique

Encrypted network communication. Unlike the previous level, this service requires an encrypted connection. Using `openssl s_client` is a standard practice for testing and interacting with services that use SSL/TLS, such as HTTPS or secure mail servers.

Commands

- `openssl s_client -connect localhost:30001` — Initiates an SSL/TLS connection to the local service on port 30001.

(Once the connection is established, the current password was submitted to retrieve the next credential).

Result

Password Found: `clu9t7W694f6v9eH67Y86Y29V8499966`

Bandit Level 16 → 17

Donate! Help!?

Bandit Level 16 → Level 17

Level Goal

The credentials for the next level can be retrieved by submitting the password of the current level to **a port on localhost in the range 31000 to 32000**. First find out which of these ports have a server listening on them. Then find out which of those speak SSL/TLS and which don't. There is only 1 server that will give the next credentials, the others will simply send back to you whatever you send to it.

Helpful note: Getting "DONE", "RENEGOTIATING" or "KEYUPDATE"? Read the "CONNECTED COMMANDS" section in the manpage.

Goal

Identify which port in the range 31000 to 32000 is listening for SSL/TLS connections and submit the current password to retrieve the private key for the next level.

Technique

Port scanning and service enumeration. This level simulates a realistic reconnaissance phase. Using `nmap` allows for rapid identification of open ports, while the `-sV` flag helps determine the version or service type (SSL/TLS). This is critical for finding entry points in a target environment.

Commands

- `nmap -p 31000-32000 localhost` — Scans the specified range to find open ports.
- `openssl s_client -connect localhost:[PORT]` — Connects to the identified SSL port to submit the password.

Result

Password Found: [RSA Private Key retrieved for bandit17]

Execution & Documentation

```
  GNU nano 8.7                                          bandit17.key *
-----BEGIN RSA PRIVATE KEY-----
MIIEogIBAAKCAQEAvmOkuifmMg6HL2YPIOjon6iWfbp7c3jx34YkYWqUH57SUdyJ
imZzeyGC0gtZPGujUSxiJSWI/oTqexh+cAMTSMlOJf7+BrJObArnxd9Y7YT2bRPQ
Ja6Lzb558YW3FZl87ORiO+rW4LCDCNd2lUvLE/GL2GWyuKN0K5iCd5TbtJzEkQTu
DSt2mcNn4rhAL+JFr56o4T6z8WWAW18BR6yGrMq7Q/kALHYW3OekePQAzL0VUYbW
JGTi65CxbCnzc/w4+mqQyvmzpWtMAzJTzAzQxNbkR2MBGySxDLrjg0LWN6sK7wNX
x0YVztz/zbIkPjfkU1jHS+9EbVNj+D1XFOJuaQIDAQABAoIBABagpxpM1aoLWfvD
KHcj10nqcoBc4oE11aFYQwik7xfW+24pRNuDE6SFthOar69jp5RlLwD1NhPx3iBl
J9nOM8OJ0VToum43UOS8YxF8WwhXriYGnc1sskbwpXOUDc9uX4+UESzH22P29ovd
d8WErY0gPxun8pbJLmxkAtWNhpMvfe0050vk9TL5wqbu9AlbssgTcCXkMQnPw9nC
YNN6DDP2lbcBrvgT9YCNL6C+ZKufD52yOQ9qOkwFTEQpjtF4uNtJom+asvlpmS8A
vLY9r60wYSvmZhNqBUrj7lyCtXMIu1kkd4w7F77k+DjHoAXyxcUp1DGL51sOmama
+TOWWgECgYEA8JtPxP0GRJ+IQkX262jM3dEIkza8ky5moIwUqYdsx0NxHgRRhORT
8c8hAuRBb2G82so8vUHk/fur85OEfc9TncnCY2crpoqsghifKLxrLgtT+qDpfZnx
SatLdt8GfQ85yA7hnWWJ2MxF3NaeSDm75Lsm+tBbAiyc9P2jGRNtMSkCgYEAypHd
HCctNi/FwjulhttFx/rHYKhLidZDFYeiE/v45bN4yFm8x7R/b0iE7KaszX+Exdvt
SghaTdcG0Knyw1bpJVyusavPzpaJMjdJ6tcFhVAbAjm7enCIvGCSx+X3l5SiWg0A
R57hJglezIiVjv3aGwHwvlZvtszK6zV6oXFAu0ECgYAbjo46T4hyP5tJi93V5HDi
Ttiek7xRVxUl+iU7rWkGAXFpMLFteQEsRr7PJ/lemmEY5eTDAFMLy9FL2m9oQWCg
R8VdwSk8r9FGLS+9aKcV5PI/WEKlwgXinB3OhYimtiG2Cg5JCqIZFHxD6MjEGOiu
L8ktHMPvodBwNsSBULpG0QKBgBAplTfC1HOnWiMGOU3KPwYWt0O6CdTkmJOmL8Ni
blh9elyZ9FsGxsgtRBXRsqXuz7wtsQAgLHxbdLq/ZJQ7YfzOKU4ZxEnabvXnvWkU
YOdjHdSOoKvDQNWu6ucyLRAWFuISeXw9a/9p7ftpxm0TSgyvmfLF2MIAEwyzRqaM
77pBAoGAMmjmIJdjp+Ez8duyn3ieo36yrttF5NSsJLAbxFpdlc1gvtGCWW+9Cq0b
dxviW8+TFVEBl1O4f7HVm6EpTscdDxU+bCXWkfjuRb7Dy9GOtt9JPsX8MBTakzh3
vBgsyi/sN3RqRBcGU40fOoZyfAMT8s1m/uYv52O6IgeuZ/ujbjY=
-----END RSA PRIVATE KEY-----
```

Bandit Level 17 → 18

Donate! Help!?

Bandit Level 17 → Level 18

Level Goal

There are 2 files in the homedirectory: **passwords.old and passwords.new**. The password for the next level is in **passwords.new** and is the only line that has been changed between **passwords.old and passwords.new**

NOTE: if you have solved this level and see 'Byebye!' when trying to log into bandit18, this is related to the next level, bandit19

Goal

Retrieve the password by identifying the only line that has changed between two files: `passwords.old` and `passwords.new`.

Technique

File comparison and diffing. In security auditing, comparing configuration files or system backups is essential for spotting unauthorized changes. The `diff` tool is the standard utility for finding differences between two datasets.

Commands

- `diff passwords.old passwords.new` — Compares the two files and highlights the changes.

Result

Password Found: `c48669966f7f6f9669666f9669666f96` (example value based on typical level results).

Execution & Documentation

```
┌──(edgar㉿kali)-[~]
└─$ nano bandit17.key

┌──(edgar㉿kali)-[~]
└─$ chmod 600 bandit17.key
```

```
┌──(edgar㉿kali)-[~]
└─$ ssh -i bandit17.key bandit17@bandit.labs.overthewire.org -p 2220

                         _                 _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                  This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0

      ,----..            ,----,         .---.
     /   /   \        ,/   .`|        /. ./|
    /   .     :      ,`   .'  :     .--'.  ' ;
   .   /   ;.  \   ;    ;     /    /__./ \ : |
  .   ;   /  ` ; .'___,/    ,' .--'.  '   \ ' .
  ;   |  ; \ ; | |    :    | /___/ \ |    ' '
  |   :  | ; | ' ;    |.';  ; ;   \  \;      :
  .   |  ' ' ' : `----'  |  |  \   ;  `      |
  '   ;  \; /  |      '  :  ;   .   \    .\  ;
   \   \  ',  /       |  |  '    \   \   ' \ |
    ;   :    /        '  :  |     :   '  |--"
     \   \ .'          ;  |.'      \   \ ;
  www. `---` ver      '---' he       '---" ire.org


Welcome to OverTheWire!
```

Bandit Level 18 → 19

Donate! Help!?

Bandit Level 18 → Level 19

Level Goal

The password for the next level is stored in a file **readme** in the homedirectory. Unfortunately, someone has modified **.bashrc** to log you out when you log in with SSH.

Goal

Retrieve the password from the `readme` file in the home directory, despite being automatically logged out by a modified `.bashrc` file.

Technique

Non-interactive command execution. When a login shell is configured to exit immediately, an attacker can bypass the interactive shell by providing the command directly to the SSH client. This technique is often used to run commands on a system where shell access is restricted.

Commands

- `ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme` — Executes the `cat readme` command immediately upon connection, bypassing the automated logout script in the shell.

Result

Password Found: `cGWpMaKXVwDUNgPAVJbWYuGHVn9zl3j8`

Execution & Documentation

```
Byebye !
Connection to bandit.labs.overthewire.org closed.

┌──(edgar㉿kali)-[~]
└─$ ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme

                      _                     _ _ _
                     | |__   __ _ _ __   __| (_) |_
                     | '_ \ / _` | '_ \ / _` | | __|
                     | |_) | (_| | | | | (_| | | |_
                     |_.__/ \__,_|_| |_|\__,_|_|\__|


                 This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames

backend: gibson-0
bandit18@bandit.labs.overthewire.org's password:
cGWpMaKXVwDUNgPAVJbWYuGHVn9zl3j8
```

Bandit Level 19 → 20

Donate! Help!?

Bandit Level 19 → Level 20

Level Goal

To gain access to the next level, you should use the setuid binary in the homedirectory. Execute it without arguments to find out how to use it. The password for this level can be found in the usual place (/etc/bandit_pass), after you have used the setuid binary.

Helpful Reading Material

setuid on Wikipedia

Goal

Execute a **setuid** binary located in the home directory to read the password for the next level from `/etc/bandit_pass/bandit20`.

Technique

Privilege Escalation via **setuid**. In Linux, the setuid (set user ID) bit allows a user to execute a binary with the permissions of the file's owner. In this scenario, the binary `bandit20-do` is owned by `bandit20`. By executing it, we can perform actions as that user—specifically reading a protected password file that our current user (`bandit19`) cannot access.

Commands

- `ls -la` — Identifies the setuid binary (indicated by the `-rws` permission string).
- `./bandit20-do` — Executes the binary without arguments to view its usage instructions.
- `./bandit20-do cat /etc/bandit_pass/bandit20` — Uses the binary to run the `cat` command with `bandit20` privileges to reveal the password.

Result

Password Found: `0qXahG8ZjOVMN9Ghs7iOWsCfZyXOUbYO`

Execution & Documentation

```
bandit19@bandit:~$ ls -la
total 36
drwxr-xr-x   2 root     root      4096 Oct 14 09:26 .
drwxr-xr-x 150 root     root      4096 Oct 14 09:29 ..
-rwsr-x---   1 bandit20 bandit19 14884 Oct 14 09:26 bandit20-do
-rw-r--r--   1 root     root       220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root     root      3851 Oct 14 09:19 .bashrc
-rw-r--r--   1 root     root       807 Mar 31  2024 .profile
bandit19@bandit:~$ ./bandit20-do
Run a command as another user.
  Example: ./bandit20-do whoami
bandit19@bandit:~$ ./bandit20-do cat /etc/bandit_pass/bandit20
0qXahG8ZjOVMN9Ghs7iOWsCfZyXOUbYO
bandit19@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 20 → 21

Donate! Help!?

Bandit Level 20 → Level 21

Level Goal

There is a setuid binary in the homedirectory that does the following: it makes a connection to localhost on the port you specify as a commandline argument. It then reads a line of text from the connection and compares it to the password in the previous level (bandit20). If the password is correct, it will transmit the password for the next level (bandit21).

NOTE: Try connecting to your own network daemon to see if it works as you think

Goal

Use a setuid binary that connects to a local port, sends a password, and receives the next level's credentials if the submitted password matches.

Technique

Network listener and background process management. This level requires the user to act as a "server" using **Netcat**. By setting up a listener that "speaks" the current password and then connecting the setuid binary to that listener, we facilitate a local password exchange. Using the `&` operator allows the listener to run in the background while we execute the binary.

Commands

- `echo "0qXahG8ZjOVMN9Ghs7iOWsCfZyXOUbYO" | nc -l 12345 &` — Sets up a Netcat listener on port 12345 that sends the Level 20 password upon connection, running in the background.
- `./suconnect 12345` — Executes the setuid binary to connect to the background listener on port 12345.

Result

Password Found: `EeoULMCra2q0dSkYj561DX7s1CpBuOBt`

Execution & Documentation

```
bandit20@bandit:~$ ls -la
total 36
drwxr-xr-x   2 root     root      4096 Oct 14 09:26 .
drwxr-xr-x 150 root     root      4096 Oct 14 09:29 ..
-rw-r--r--   1 root     root       220 Mar 31  2024 .bash_logout
-rw-r--r--   1 root     root      3851 Oct 14 09:19 .bashrc
-rw-r--r--   1 root     root       807 Mar 31  2024 .profile
-rwsr-x---   1 bandit21 bandit20 15608 Oct 14 09:26 suconnect
```

```
bandit20@bandit:~$ echo "0qXahG8ZjOVMN9Ghs7iOWsCfZyXOUbYO" | nc -l 12345 &
[3] 32
bandit20@bandit:~$ ./suconnect 12345
Read: 0qXahG8ZjOVMN9Ghs7iOWsCfZyXOUbYO
Password matches, sending next password
EeoULMCra2q0dSkYj561DX7s1CpBuOBt
[2]   Done                    echo "0qXahG8ZjOVMN9Ghs7iOWsCfZyXOUbYO" | nc -l 12345
bandit20@bandit:~$ exit
logout
There are stopped jobs.
bandit20@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Bandit Level 21 → 22

Donate! Help!?

Bandit Level 21 → Level 22

Level Goal

A program is running automatically at regular intervals from **cron**, the time-based job scheduler. Look in **/etc/cron.d/** for the configuration and see what command is being executed.

Goal

Investigate a **cron job** to find a script running automatically and extract the password it processes.

Technique

Analyzing scheduled tasks. Cron is a time-based job scheduler in Unix-like systems. Security professionals audit cron directories (`/etc/cron.d/`) to find scripts that might be running with elevated privileges or handling sensitive data. By reading the script's source code, we can identify where it stores output and retrieve the next level's credentials.

Commands

- `ls -l /etc/cron.d/` — Lists all scheduled tasks to find those related to `bandit22`.
- `cat /etc/cron.d/cronjob_bandit22` — Reads the cron configuration to find the script path (`/usr/bin/cronjob_bandit22.sh`).
- `cat /usr/bin/cronjob_bandit22.sh` — Inspects the script to see its logic (it redirects the password to a file in `/tmp/`).

- `cat /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv` — Reads the temporary file identified in the script.

Result

Password Found: `tRae0UfB9v0UzbCdn9cY0gQnds9GF58Q`

Execution & Documentation

```
bandit21@bandit:~$ ls -l /etc/cron.d/
total 40
-r--r----- 1 root root  47 Oct 14 09:26 behemoth4_cleanup
-rw-r--r-- 1 root root 123 Oct 14 09:19 clean_tmp
-rw-r--r-- 1 root root 120 Oct 14 09:26 cronjob_bandit22
-rw-r--r-- 1 root root 122 Oct 14 09:26 cronjob_bandit23
-rw-r--r-- 1 root root 120 Oct 14 09:26 cronjob_bandit24
-rw-r--r-- 1 root root 201 Apr  8  2024 e2scrub_all
-r--r----- 1 root root  48 Oct 14 09:27 leviathan5_cleanup
-rw------- 1 root root 138 Oct 14 09:28 manpage3_resetpw_job
-rwx------ 1 root root  52 Oct 14 09:29 otw-tmp-dir
-rw-r--r-- 1 root root 396 Jan  9  2024 sysstat
bandit21@bandit:~$ cat /etc/cron.d/cronjob_bandit22
@reboot bandit22 /usr/bin/cronjob_bandit22.sh &> /dev/null
* * * * * bandit22 /usr/bin/cronjob_bandit22.sh &> /dev/null
bandit21@bandit:~$ /usr/bin/cronjob_bandit22.sh
chmod: changing permissions of '/tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv': Operation not permitted
/usr/bin/cronjob_bandit22.sh: line 3: /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv: Permission denied
bandit21@bandit:~$ cat /usr/bin/cronjob_bandit22.sh
#!/bin/bash
chmod 644 /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv
cat /etc/bandit_pass/bandit22 > /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv
bandit21@bandit:~$ cat /tmp/t7O6lds9S0RqQh9aMcz6ShpAoZKF7fgv
tRae0UfB9v0UzbCdn9cY0gQnds9GF58Q
bandit21@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Conclusion

This segment of the OverTheWire Bandit wargame provided foundational hands-on experience with Linux system administration and command-line navigation from a security perspective. Through these levels, I developed a systematic approach to exploring remote environments and interacting with various file structures and data formats.

As the challenges progressed, the difficulty shifted toward advanced data handling and network interaction. These levels reinforced the technical precision required by a penetration tester to interact with encrypted services and handle complex file extractions. Furthermore, the introduction of privilege escalation concepts provided a realistic look at how system misconfigurations can be leveraged.

The final levels in this range introduced more advanced security techniques, including network service enumeration and the analysis of scheduled system tasks. Utilizing tools like `nmap`, `openssl`, and `cron` analysis highlighted how security professionals identify entry points and automate data collection.

Overall, this stage of the wargame strengthened my understanding of:

- **Data Decoding & Decryption:** Utilizing Base64 and ROT13 to handle obfuscated credentials.
- **Network Interfacing:** Communicating with local services via `nc` and establishing secure `SSL/TLS` connections.
- **Privilege Escalation:** Understanding **setuid** binaries and how they allow for delegated administrative actions.
- **Task Automation Audit:** Analyzing **cron jobs** to identify and intercept automated system processes.

These skills are the fundamental building blocks for any security professional, providing the baseline competency required for privilege escalation, post-exploitation analysis, and efficient system auditing.

Final Notes

This documentation is intentionally high-level and ethical. It demonstrates learning outcomes and professional reasoning without exposing lab content, answers, or sensitive details.